



جامعة دمشق
كلية الهندسة المعلوماتية
السنة الرابعة - اختصاص الذكاء الصناعي



مشروع خوارزميات البحث الذكية senet game play

إعداد وتقديم الطلاب:

يمان غسان بسطاوي

محمد منير محمد الكنفاني

محمد عمار محمد ماهر دهان

يمان أنس الطحان

أولاً: نبذة سريعة عن اللعبة:

لعبة senet هي لعبة رقعة تحتوي على 30 خلية يبدأ اللاعبان بـ 7 أحجار لكل لون (أبيض و أسود) اللاعب الذي يخرج جميع أحجاره أولاً يربح اللعبة

يخرج الحجر من الرقعة عندما يصل للخلية رقم 30

يتحرك اللاعب بناءً على عدد العصي:

ظهور من 1 إلى 4 عصي عدد الخطوات = عدد العصي

ظهور 0 عدد الخطوات = 5

ملاحظة يجب التقيد بعدد الخطوات الناتج عن العصي تماماً في الخطوة

ترتيب الخلايا على الشكل الآتي :

1	2	3	4	5	6	7	8	9	10
20	19	18	17	16	15	14	13	12	11
21	22	23	24	25	26	27	28	29	30

البيوت (الخلايا) المميزة :

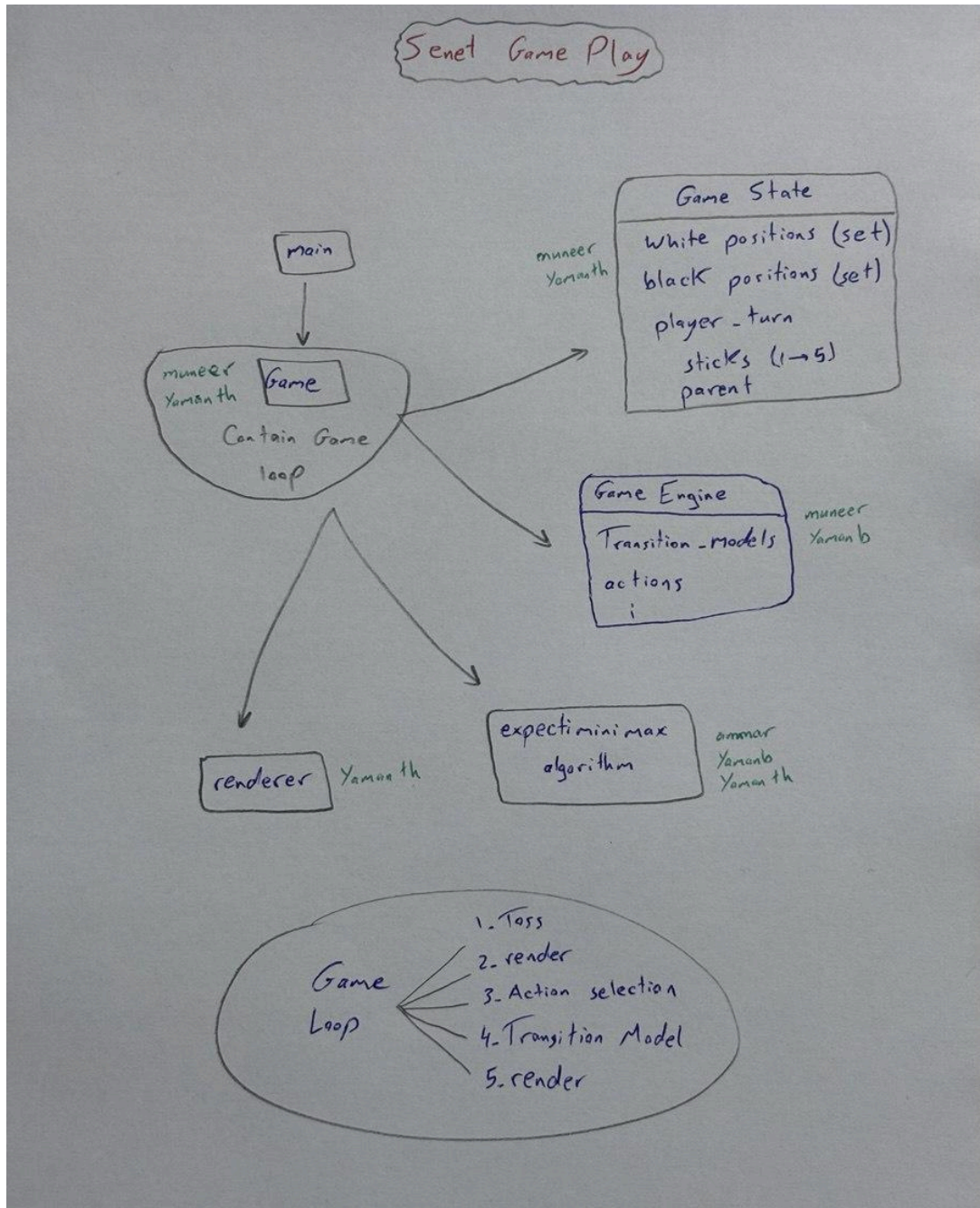
هي الخلايا التي لها تأثيرات تجعل اللعبة تنافسية واستراتيجية:

1. الخلية رقم 15 ليس لها تأثير فعلياً ولكن ترجع لها الأحجار في بعض الحالات وإذا كانت مشغولة تعود للحجر السابق
2. الخلية 26 لا يمكن تجاوزها يجب الوقوف عندها
3. الخلية 27 إذا وقف حجر عليها فسيعود مباشرة إلى 15
4. الخلية رقم 28 في الدور التالي عند سحب عصا في حال ظهور غير 3 ترجع إلى الخلية 15
5. الخلية رقم 29 في الدور التالي عند سحب عصا في حال ظهور غير 2 ترجع إلى الخلية 15
6. الخلية 30 هي الخلية النهائية حال الوصول إليها في الدور التالي يمكن الخروج من الرقعة في حال عدم الخروج ستعيد الحجر إلى الخلية 15

ثانياً: التخطيط الجماعي:

هذه هي الخطة الأولية للعمل على المشروع.

ملاحظة لم نتقيد بها بشكل كامل بعض الأشياء تغيرت أثناء العمل.



ثالثاً : المخطط النهائي للمشروع:

لدينا خمس أقسام رئيسية :

1. القسم الأول تمثيل المسألة :

يحتوي على البيئة ومنطق اللعبة , يمثل بالملفات: (game_state + game_engine).

2. القسم الثاني قسم الخوارزمية:

نطبق في هذا القسم الخوارزمية المطلوبة ويمثل بالملفات: (expectiminimax).

3. القسم الثالث قسم الجرافيك:

في هذا القسم نستعمل مكتبة pygame لتمثل اللعبة بصرياً ويمثل بالملفات: (renderer).

4. القسم الرابع قسم الأطوار:

في هذا القسم نعالج أطوار اللعب:

- لاعب ضد لاعب.
- لاعب ضد خوارزمية.
- خوارزمية ضد خوارزمية.

5. القسم الخامس قسم game flow:

في هذا القسم نعالج سير اللعبة بتبديل الأدوار أي فعليا استدعاء باقي الكلاسات يتضمن ملفات: (class diagram + flowchart diagram).

القسم الأول: تمثيل اللعبة + منطق اللعبة:

تمثيل المسألة:

في هذه المسألة لتمثيل اللعبة فإننا نحتاج إلى تمثيل كل من اللاعبين (أسود , أبيض).
بالإضافة إلى أماكن أحجار كل لاعب في الحالة الافتراضية.
أيضاً يجب تمثيل العصي.

```
from enum import Enum, auto

class PlayerColor(Enum):
    BLACK = auto()
    WHITE = auto()

class State:
    """
    the Game State contains:
    white positions (set),
    black positions (set),
    player turn (PlayerColor),
    sticks (int 1 -> 5),
    parent (state)
    """

    def __init__(self, white_positions:set = None, black_positions:set = None, current_player:PlayerColor = PlayerColor.BLACK, sticks:int = 5, parent:State = None):
        self.white_positions = white_positions if white_positions is not None else set({1, 3, 5, 7, 9, 11, 13})
        self.black_positions = black_positions if black_positions is not None else set({2, 4, 6, 8, 10, 12, 14})
        self.current_player = current_player
        self.sticks = sticks
        self.parent = parent

    def change_player(self):
        next_player = PlayerColor.BLACK if self.current_player == PlayerColor.WHITE else PlayerColor.WHITE
        self.current_player = next_player
        self.sticks = 0
```

في هذا الكلاس State قمنا بتمثيل اللعبة.

تم اختيار set لتمثيل أحجار كل لاعب.

في الحالة الابتدائية يكون اللاعب الأبيض في المواقع التالية {1,3,5,7,9,11,13}.

أما اللاعب الأسود يبدأ في المواقع التالية {2,4,6,8,10,12}.

أما لون اللاعب فهو اينوم قيمتين إما أبيض أو أسود.

بالنسبة للتابع chaneg_player():

يقوم بتبديل الدور عن طريق تبديل لون اللاعب.

يتم تصفير العصي عند كل دور جديد (عند بداية الدور قبل رمي العصا يكون صفر).

منطق اللعبة:

هذا الكلاس يمثل فعلياً عقل اللعبة يحتوي على عدة توابع رئيسية:

أولاً : تابع سحب العصي `TossStick()`:

يقوم باختيار عدد عصي عشوائياً لكن بناءً على احتمالات محددة.

بايثون تقدم تابع جاهز يقوم بهذه المهمة وهو التابع `choice` يأخذ باراميتران الأول الاختيارات الثاني احتمال كل خيار.

في حال كان عدد العصي 0 نرجع 5 (عدد الخطوات).

```
def TossStick(self):
    """This function return int (1 -> 5) probability of sticks"""
    sticks = [0, 1, 2, 3, 4]
    probs = [1/16, 4/16, 6/16, 4/16, 2/16]
    choice = random.choices(sticks, probs)[0]
    return 5 if choice == 0 else choice
```

ثانياً : تابع `is_action_valid()`:

`action` تعبر عن الحجر المراد تحريكه.

هذا التابع يقوم بالتأكد من أن الحجر المراد تحريكه مطابق للون اللاعب الحالي بالدور.

```
def _is_valid_action(self, state: State, action: int) -> bool:
    """
    Check is action is valid for the current player
    To prevent collision between pawns from the same color
    """
    if state.current_player == PlayerColor.BLACK:
        return action in state.black_positions
    else:
        return action in state.white_positions
```

ثالثاً: تابع actions():

هذا التابع يقوم بإرجاع قائمة تحتوي الخطوات المتاحة.

يقوم بعمل تكرار على كافة أحجار اللاعب والتأكد من الأحجار الممكن تحريكها في حال يمكن تحريكه سيتم إضافته للسيت.

أول شرط يقوم بالتحقق من أن الحجر لن يتحرك لمكان حجر آخر لنفس اللاعب.

ثاني شرط في حال الخطوة قبل 27 لن تتجاوز أو تقف على الخلية 27 يسمح بالحركة (لأن 26 تمنع تجاوزها).

ثالث شرط في حال كنا نقف على 26 فيمكن التحرك.

رابع شرط يمكننا التحرك من الخلية 28 فقط إذا كان عدد العصي 3.

خامس شرط يمكننا التحرك من الخلية 29 إذا كان عدد العصي 2.

الشرط الأخير يمكننا التحرك في حال كنا على الخلية 30.

```
def actions(self, state:State):
    """
    This function return a set of pawns that player can move
    e.g: set (1, 3, 5, 7, 9, 11, 13) the pawns and sticks = 2
    """
    return set(13) the available moves

    """
    current_positions = set()
    movable_positions = set()
    if state.current_player == PlayerColor.BLACK:
        current_positions = state.black_positions
    else:
        current_positions = state.white_positions
    for current_position in current_positions:
        new_position = current_position + state.sticks
        if new_position in current_positions:
            continue
        elif new_position < 27:
            movable_positions.add(current_position)
        elif current_position == 26:
            movable_positions.add(current_position)
        elif current_position == 28 and state.sticks == 3:
            movable_positions.add(current_position)
        elif current_position == 29 and state.sticks == 2:
            movable_positions.add(current_position)
        elif current_position == 30:
            movable_positions.add(current_position)
    return movable_positions
```

رابعاً: تابع move_pawn():

هذا التابع يحرك الحجر في:

الشرط الأول لن يسمح بتجاوز 26.

الشرط الثاني لن يسمح بالحركة إلى خلية مشغولة من نفس اللاعب.

الشرط الثالث في حال كانت الحركة تضع الحجر على الخلية 27 سنرجع الحجر مباشرة إلى الخلية 15 أو أول خلية فارغة سابقة.

في حال الخلايا ما قبل 27 أي لن تتجاوز 26 لدينا احتمالان نريد التحرك الى خلية فارغة فسيتم التحريك أو خلية مشغولة من قبل اللاعب الخصم سنقوم بالتبديل.

الشرط الرابع إذا كان الحجر المراد تحريكه يقف على الخلية 26 أو بعد قم بتحريكه (لا يوجد تبديل لأن هذه الخلايا لن تقف عليها الاحجار أكثر من دور).

وإذا كانت الحجر ذاهب الى خلية ضمن الرقعة أصغر من 3 قم بإضافته.

```
def _move_pawn(self, action, current_player_positions, opponent_positions, sticks):
    """Move pawn and handle switching"""
    new_position = action + sticks
    path = range(action + 1, sticks + action + 1)
    if 26 in path and action + sticks > 26:
        return False
    if new_position in current_player_positions:
        return False
    elif new_position == 27:
        current_player_positions.remove(action)
        new_pos = self._first_previous(15, current_player_positions | opponent_positions)
        current_player_positions.add(new_pos)
    elif new_position < 27:
        current_player_positions.remove(action)
        current_player_positions.add(new_position)
        if new_position in opponent_positions:
            opponent_positions.remove(new_position)
            opponent_positions.add(action)
    elif action >= 26:
        current_player_positions.remove(action)
        if new_position < 31:
            current_player_positions.add(new_position)
    else:
        return False
    return True
```

خامساً: تابع `first_previous()`:

هذا التابع يعالج حالة عند إرجاع الحجر للخلية 15 ولكن الخلية 15 مشغولة بحجر آخر فإن هذا التابع يبحث عن أول خلية فارغة سابقة لإرجاع الحجر لها.

```
def _first_previous(self, action, positions):
    """This function returns the first empty cell less than 16"""
    for position in range(action, 0, -1):
        if position not in positions:
            return position
    return None
```

سادساً: تابع `_handle_special_houses()`:

هذا التابع يقوم بمعالجة الخلايا المميزة 27 28 29.

إذا كان الحجر على الخلية 28 ولم يحصل على ثلاث عصي أو لم يحركها سوف يرجع إلى 15 أو أول خلية فارغة قبلها.

إذا كان الحجر على 29 ولم يحصل على عصايتين أو لم يحركها سوف يرجع إلى 15 أو أول خلية فارغة قبلها.

إذا كان على 30 ولم نحركه سوف يرجع إلى 15 أو أول خلية فارغة قبلها.

```
def _handle_special_houses(self, current_player_positions, opponent_positions, sticks, action):
    """Handling 28, 29, 30 Houses"""
    occupied_positions = current_player_positions | opponent_positions
    for pawn in current_player_positions:
        if (pawn == 28 and (sticks != 3 or action != 28)) or \
            (pawn == 29 and (sticks != 2 or action != 29)) or \
            (pawn == 30 and action != 30):
            new_position = self._first_previous(15, occupied_positions)
            current_player_positions.remove(pawn)
            current_player_positions.add(new_position)
```

سابعاً: تابع ()model_transaction:

هذا التابع هو حجر أساس اللعب حيث يقوم بتطبيق الحركة المرادة على الحالة الحالية.

```
class GameEngine:
    def transition_model(self, state:State, action:int):
        """
        Transition to a new state by moving a pawn
        """
        if action is None or state.sticks == 0:
            return None
        if not self._is_valid_action(state, action):
            return None
        white_positions:set = state.white_positions.copy()
        black_positions:set = state.black_positions.copy()
        sticks:int = state.sticks
        current_player = state.current_player
        if (action == 12 and sticks == 3) or (action == 13 and sticks == 2):
            # handle pawn move
            if current_player == PlayerColor.BLACK:
                success = self._move_pawn(action, black_positions, white_positions, sticks)
            else:
                success = self._move_pawn(action, white_positions, black_positions, sticks)
            if not success:
                return None
            # handle special houses
            if current_player == PlayerColor.BLACK:
                self._handle_special_houses(black_positions, white_positions, sticks, action)
            else:
                self._handle_special_houses(white_positions, black_positions, sticks, action)
        else:
            # handle special houses
            if current_player == PlayerColor.BLACK:
                self._handle_special_houses(black_positions, white_positions, sticks, action)
            else:
                self._handle_special_houses(white_positions, black_positions, sticks, action)
            # handle pawn move
            if current_player == PlayerColor.BLACK:
                if current_player == PlayerColor.BLACK:
                    success = self._move_pawn(action, black_positions, white_positions, sticks)
                else:
                    success = self._move_pawn(action, white_positions, black_positions, sticks)
                if not success:
                    return None
            # switch players
            next_player = PlayerColor.BLACK if current_player == PlayerColor.WHITE else PlayerColor.WHITE
            return State(
                white_positions,
                black_positions,
                next_player,
                0,
                state
            )
```

القسم الثاني: الخوارزمية:

تم تطبيق الخوارزمية ضمن كلاس ExpectiMinimaxPlayer وسنناقش آلية عمل التوابع الأساسية فيه:

1. احتمالات رمي العصي :

في بداية الكلاس قمنا بتعريف مصفوفة الاحتمالات self.probabilities بما اننا نرمي 4 عصي فهناك 16 حالة و 5 احتمالات (كما هو موضح بالصورة)

قمنا بحساب احتمال كل نتيجة (عدد الخطوات من 1 إلى 5) على التوزيع الاحتمالي لظهور الوجوه وهو أمر ضروري لحساب عقدة الحظ (Chance Node)

```
def __init__(self, depth: int):  
    self.depth = depth  
    self.action = None  
    self.game_engine = GameEngine()  
    self.probabilities = {1: 4/16, 2: 6/16, 3: 4/16, 4: 1/16, 5: 1/16}
```

2. التابع الرئيسي للبحث (eminimax) :

قمنا بكتابة تابع شامل بعالج ثلاث انواع مختلفة من العقد (Nodes) عبر استدعاء ذاتي (Recursive):

- عقدة الحظ (TOSS Node): عندما تكون العقدة من نوع TOSS سنقوم بحساب القيمة المتوقعة (Expected Value)
- نقوم بتوليد حالات متوقعة لكل احتمال من ال 5، ثم نقوم باستدعاء عودي حسب اللاعب إما min أو max وبعد حساب قيمة ال evaluate لكل عقدة، نقوم بضرب كل قيمة مع احتمالها ونجمع النتائج حتى نصل إلى expected_value.
- عقدة اللاعب (MAX Node) : تمثل دور الخوارزمية حيث نبحث عن الحركة التي تعطينا أعلى قيمة متوقعة (expected_value) ممكنة.
- عقدة الخصم (MIN Node) : تمثل دور الخصم حيث نفترض أنه سيلعب بذكاء ويختار الحركة التي تعطينا أقل قيمة بالتالي نختار أقل قيمة متوقعة للخصم

```

def minimax(self, state:State, depth:int, node:str, indent=0, is_root=False) -> float: # type: ignore
    print(" " * indent, node)
    possible_moves = self.game_engine.actions(state)

    if depth == 0 or state.is_terminal():
        return self.evaluate(state)

    if node == "TOSS":
        expected = 0
        for sticks, prob in self.probabilities.items():
            toss_state = deepcopy(state)
            toss_state.sticks = sticks

            next_node = "MAX" if state.current_player == PlayerColor.BLACK else "MIN"

            value = self.minimax(toss_state, depth - 1, next_node, indent + 1)

            expected += prob * value

        return expected

    # if the node is MAX
    elif node == "MAX":
        best_value = -inf
        best_action = None
        for move in possible_moves:
            next_state = self.game_engine.transition_model(state, move)

            if next_state is None:
                continue

            value = self.minimax(next_state, depth, "TOSS", indent + 1)

            if value > best_value:
                best_value = value
                best_action = move

        if is_root:
            return best_action

        return best_value

    # if the node is MIN
    elif node == "MIN":
        best_value = inf
        best_action = None
        for move in possible_moves:
            next_state = self.game_engine.transition_model(state, move)

            if next_state is None:
                continue

            value = self.minimax(next_state, depth, "TOSS", indent + 1)

            if value < best_value:
                best_value = value
                best_action = move

        if is_root:
            return best_action

        return best_value

```

3. سياسة التقييم (Heuristic Evaluation):

بما أننا لا نستطيع الوصول لنهاية الشجرة دائما بسبب ضخامة الاحتمالات كان لا بد من بناء تابع التقييم **evaluate** لتقدير جودة الحالة عند الوصول لعمق معين

تعتمد استراتيجيتنا في التقييم على :

- حالة الفوز / الخسارة : إذا وصلت الحالة لنهاية اللعبة ترجع قيمة كبيرة جدا (10000) في حال فوز اللاعب الأسود وقيمة سالبة جدا في حال فوز اللاعب الابيض
 - النقاط التراكمية : نمنح نقاط لكل حجر بناء على تقدمه في المربعات
 - البيوت الخاصة : قمنا بإعطاء أوزان خاصة للمربعات الاستراتيجية
- مثل : بيت الماء (27) له وزن سالب -50 لانه يعيد اللاعب للخلف بينما بيت السعادة (26) له وزن ايجابي 40+
- إخراج الاحجار : نمنح 100 نقطة إضافية عن كل حجر يخرج من الرقعة

```
def _get_player_score(self, positions:set) -> float:
    score = 0.0
    House_of_Happiness = 26
    House_of_Water = 27
    House_of_Three = 28 # square you can get stuck in
    House_of_Atom = 29 # square you can get stuck in
    Total_Pawns = 7
```

```
    for pawn_pos in positions:
        if pawn_pos == House_of_Water:
            | score -= 50

        elif pawn_pos == House_of_Happiness:
            | score += 40
        elif pawn_pos == House_of_Three:
            | score -= 25
        elif pawn_pos == House_of_Atom:
            | score -= 30
        else:
            | score += pawn_pos
```

```
    pawns_off_board = Total_Pawns - len(positions)
    score += pawns_off_board * 100
```

```
    return score
```

```
def evaluate(self, state: State) -> float:
```

```
    if state.winner() == "Black":
        | return 10000
    elif state.winner() == "White":
        | return -10000
```

```
    black_score = self._get_player_score(state.black_positions)
    white_score = self._get_player_score(state.white_positions)
```

```
    return black_score - white_score
```

4. اختيار الحركة الافضل (Best Action) :

للقيام بالحركة يستدعي كلاس ال game التابع best_action الذي يستدعي إما min او max حسب دور اللاعب الحالي (خصيصاً في وضع خوارزمية ضد خوارزمية)

قمنا بضبط العمق في الكود ليكون 2 وهو عمق كاف ليعطي أداء ذكي وسريع

حيث يتم احتساب آلاف الاحتمالات في ثوان معدودة قبل اختبار الحجر الأمثل لتحريكه

```
def best_action(self, state:State):  
    if state.current_player == PlayerColor.BLACK:  
        return self.best_action_max(state)  
    elif state.current_player == PlayerColor.WHITE:  
        return self.best_action_min(state)  
  
def best_action_max(self, state:State):  
    action = self.eminimax(state, 2, "MAX", is_root=True)  
    return action  
  
def best_action_min(self, state:State):  
    action = self.eminimax(state, 2, "MIN", is_root=True)  
    return action
```

ملاحظات :

استخدمنا مكتبة deepcopy لضمان عدم التلاعب بالحالة الأصلية للعبة اثناء تجربة الاحتمالات في الشجرة

اضفنا indent في تابع البحث لمساعدتنا اثناء عملية ال (Debugging) لطباعة الشجرة بشكل مرئي في ال console تتبع الخوارزمية

القسم الثالث: اطوار اللعبة:

تحتوي اللعبة على أربعة أطوار أساسية وهي:

1. طور اللاعبين.
2. طور اللاعب ضد الذكاء الاصطناعي.
3. طور الذكاء الاصطناعي ضد الذكاء الاصطناعي.
4. طور الذكاء الاصطناعي ضد الاختيار العشوائي.

لدينا أربع ملفات أساسية:

أولاً: **player_mode.py**:

في طور اللاعبين نقوم بشكل أساسي بمعالجة ضغطة الماوس

حيث أن الضغط على الماوس يحتمل حالتين (-1) أي يريد سحب عصا

أما في حال الضغط على حجر سيتم إرساله ك **action**.

تابع **update**: هو المحرك لهذا الطور حيث يتأكد من وجود حجر يراد تحريكه ثم يقوم بتطبيق الحركة عليه، وأخيراً تحديث الحالة إلى الحالة التالية.

```
odes > player_mode.py > PlayerMode > processInput
1 import pygame
2
3 class PlayerMode:
4     """Class that contain game loop (player vs player)"""
5     def __init__(self, game):
6         self.game = game
7
8     def processInput(self, events):
9         for event in events:
10             if event.type == pygame.MOUSEBUTTONDOWN:
11                 click_value = self.game.renderer.handle_events(
12                     event.button,
13                     self.game.state.sticks,
14                     self.game.state,
15                     self.game.engine.actions(self.game.state)
16                 )
17                 if click_value == None:
18                     return
19                 elif click_value == -1:
20                     sticks = self.game.engine.TossStick()
21                     self.game.state.sticks = sticks
22                     self.game.engine.handle_special_houses(self.game.state)
23                 else:
24                     self.game.action = click_value
25
26     def update(self):
27         if self.game.action is not None:
28             new_state = self.game.engine.transition_model(self.game.state, self.game.action)
29             if new_state is not None:
30                 self.game.state = new_state
31                 self.game.action = None
32
```

ثانياً: طور اللاعب ضد الخوارزمية:

يمثل ضمن ملف: algo_player_mode:

تابع معالجة الضغط نفسه لكن يعتمد على اللاعب الأبيض فقط.

في تابع update أولاً نرى هل اللاعب الحالي هو بشري أم ذكاء اصطناعي في حال عدم وجود حركات متاحة يتم تغيير الدور.

تابع algorithm turn إذا كان دور الذكاء سيقوم باختيار أفضل حركة باستخدام الخوارزمية إذا كانت الحركة صالحة سوف يقوم بتحريكها.

أما بالنسبة لتابع player-turn فقط سيقوم بتطبيق الحركة على الحجر الذي ضغط عليه اللاعب.

```
def update(self):
    if self.game.state.current_player == PlayerColor.BLACK:
        self._algorithm_turn()
    elif self.game.state.current_player == PlayerColor.WHITE:
        self._player_turn()

def _algorithm_turn(self):
    sticks = self.game.engine.TossStick()
    self.game.state.sticks = sticks
    self.game.engine.handle_special_houses(self.game.state)

    if not self.game.engine.actions(self.game.state):
        self.game.state.change_player()
        return

    best_action = self.algo.best_action(self.game.state)

    if best_action and best_action is not None:
        self.game.action = best_action
        new_state = self.game.engine.transition_model(self.game.state, self.game.action)
        if new_state is not None:
            self.game.state = new_state
            self.game.action = None

def _player_turn(self):
    new_state = self.game.engine.transition_model(self.game.state, self.game.action)
    if new_state is not None:
        self.game.state = new_state
        self.game.action = None
```

ثالثاً: الذكاء الاصطناعي ضد الذكاء الاصطناعي:

هذا التابع يقوم باتمته اللعب حيث كلا اللاعبين ai.

يختار أفضل حركة باستدعاء الخوارزمية على اللاعب الحالي (أبيض/أسود)
ثم يطبق هذه الحركة في حال كانت صالحة وتجديد الحالة الحالية.

```
def _algorithm_turn(self, ai_player):
    sticks = self.game.engine.TossStick()
    self.game.state.sticks = sticks
    self.game.engine.handle_special_houses(self.game.state)

    if not self.game.engine.actions(self.game.state):
        self.game.state.change_player()
        return

    ai_state = deepcopy(self.game.state)

    best_action = ai_player.best_action(ai_state)

    if best_action and best_action is not None:
        self.game.action = best_action
        new_state = self.game.engine.transition_model(self.game.state, self.game.action)
        if new_state is not None:
            self.game.state = new_state
            self.game.action = None
```

رابعاً: الذكاء ضد لاعب عشوائي:

نفس الحالة السابقة الفرق أن اللاعب الأبيض يختار الحركة عشوائياً بدلاً من استدعاء الخوارزمية.

```
def _random_agent_turn(self):
    sticks = self.game.engine.TossStick()
    self.game.state.sticks = sticks
    self.game.engine.handle_special_houses(self.game.state)

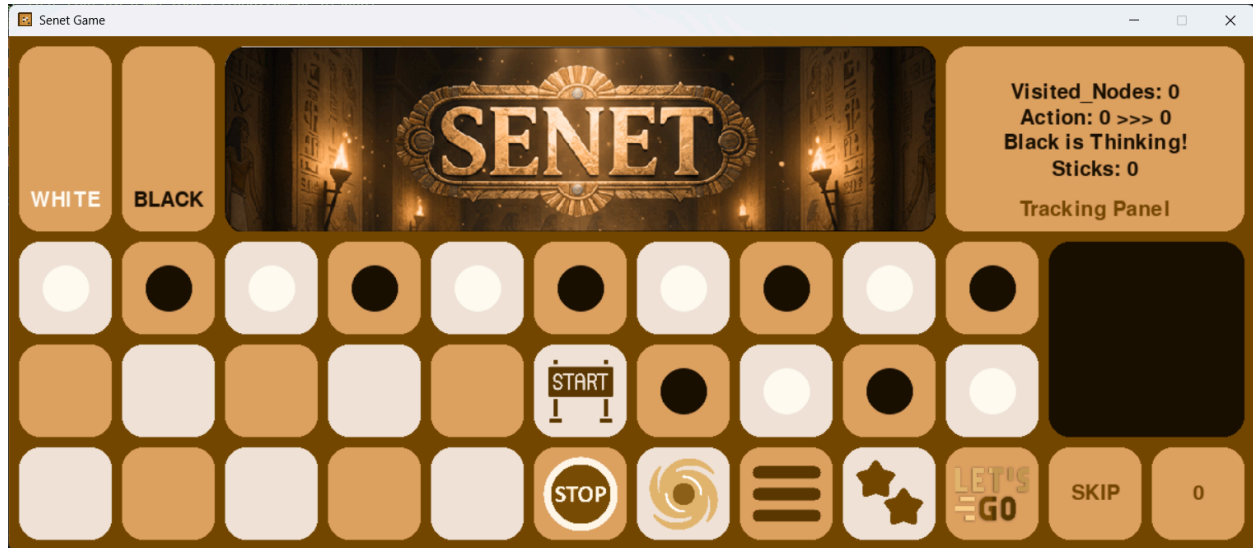
    actions = self.game.engine.actions(self.game.state)
    if not actions:
        self.game.state.change_player()
        return

    # best_action = ai_player.find_best_move(self.game.state)
    action = random.choice(list(actions))

    if action and action is not None:
        self.game.action = action
        new_state = self.game.engine.transition_model(self.game.state, self.game.action)
        if new_state is not None:
            self.game.state = new_state
            self.game.action = None
```

رابعاً: القسم الرابع قسم الجرافيك

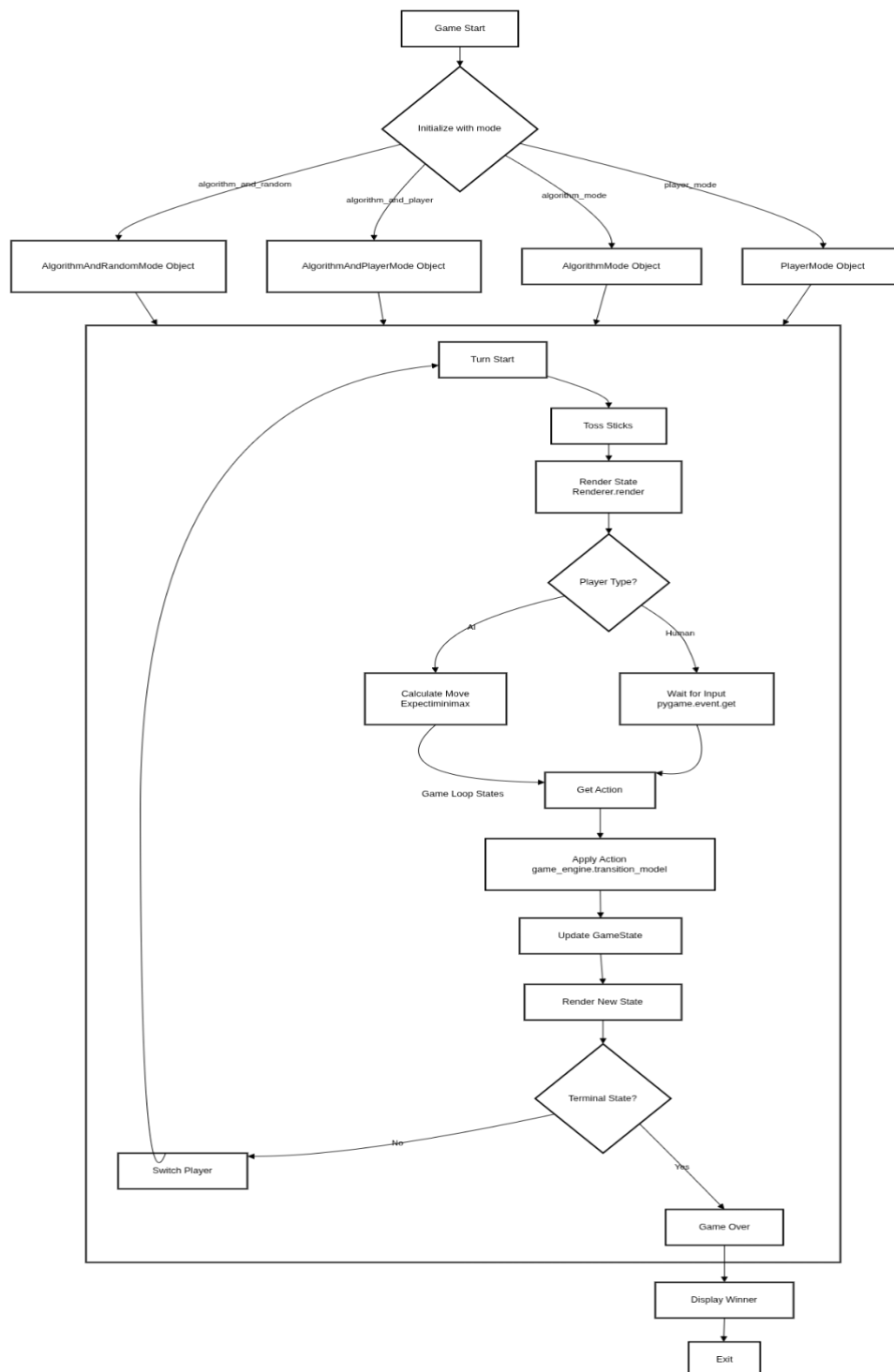
- اعتمدنا بشكل رئيسي على مكتبة pygame للإخراج البصري في المشروع



- يوجد كلاس Renderer يحوي توابع لعرض كل عنصر على حدى (grid, special)
- تم الاعتماد على وحدة عرض أساسية و التي تمثل مربع بمقاس ثابت وبعدها تم إنشاء باقي العناصر بالاعتماد على هذه الوحدة الرئيسية .

القسم الخامس: إدارة تدفق اللعبة :game flow

FlowChart Diagram:



Class Diagram:

